

المعهد الهندي للتكنولوجيا دلهي
أبوظبي

INDIAN INSTITUTE OF TECHNOLOGY DELHI
ABU DHABI



AMTL106 Assignment 2 Report

Queuing Theory Analysis Using Simulations

Authors: Mohammad Taqi Ahsan, Abhiramachandra Vemparala

Date Submitted: 1st December 2025

Instructor: Prof. Rahul Singh

Academic Year: 2025–26

Probability and Stochastic Processes
Indian Institute of Technology Delhi - Abu Dhabi

Contents

1	Introduction	3
2	Theoretical Background	4
2.1	Poisson Arrival Process	4
2.2	Exponential Service Times	4
2.3	Markov Chains and Birth–Death Processes	4
2.4	M/M/1 Queue Formulas	4
2.5	M/M/c Queue Formulas (Erlang C)	5
2.6	Finite Capacity Queue M/M/1/K	5
3	Analytical Results (Numeric)	6
3.1	Case 1: M/M/1 (Toll booth, 1 server)	6
3.2	Case 2: M/M/2 (Toll booth, 2 servers)	6
3.3	Case 3: M/M/1/K (Café, $K = 10$)	7
4	Simulation Methodology	8
4.1	Python Code Snippet for M/M/1 and M/M/c Simulations	8
4.2	Python Code Snippet for M/M/1/k Simulation	12
5	Simulation Results	16
5.1	Toll Booth with 1 Server (M/M/1)	16
5.2	Toll Booth with 2 Servers (M/M/c)	16
5.3	Café with Limited Queue Capacity (M/M/1/K)	17
6	Comparison: Theoretical vs Simulated Results	19
7	Discussion	19
7.1	Case 1: M/M/1	19
7.2	Case 2: M/M/2	19
7.3	Case 3: M/M/1/K	19
8	Conclusion	20

Abstract

This project presents a comprehensive study of queueing theory through simulation-driven experimentation. We analyze three service systems using Poisson arrivals and exponential service times, and compare empirical simulation results with classical theoretical predictions.

The first case models a **single-server toll booth (M/M/1)** with arrival rate $\lambda = 10$ customers/min and service rate $\mu = 12$ customers/min. The second case extends this system to a **two-server toll booth (M/M/2)**, allowing analysis of how multiple servers affect congestion and waiting times. The third case examines a **finite-capacity café queue (M/M/1/K)** with $K = 10$, representing systems where customers may be blocked when the queue is full.

Across all three models, we evaluate system behavior in terms of utilization, waiting time, service time, and queue length, highlighting the relationship between Markov chains, birth–death processes, and practical stochastic modeling. The results demonstrate how theoretical formulas align with simulation outcomes and illustrate the operational differences between infinite-capacity, multi-server, and finite-buffer queueing systems.

1 Introduction

Queueing theory is a branch of applied probability that models systems in which customers wait for service. It has applications in computer networks, transport systems, banking, call centers, and operations management. A queueing system is typically defined by three key components:

- **Arrival process** (often Poisson)
- **Service process** (often exponential)
- **Number of servers**

In this project, we use realistic arrival and service assumptions to model three important queues:

1. Toll booth with one server – modeled as an M/M/1 queue
2. Toll booth with two servers – modeled as an M/M/2 queue
3. Café with limited queue capacity – modeled as an M/M/1/K queue

Our goals are:

- To simulate the queues using Python
- To derive analytical characteristics using queueing formulas
- To compare simulated and theoretical results

2 Theoretical Background

2.1 Poisson Arrival Process

A Poisson process is defined by:

- Independent increments
- Stationary increments
- $N(t) \sim \text{Poisson}(\lambda t)$

Interarrival times are exponential:

$$X \sim \text{Exponential}(\lambda)$$

2.2 Exponential Service Times

Service times in M/M/1-type systems follow:

$$S \sim \text{Exponential}(\mu)$$

2.3 Markov Chains and Birth–Death Processes

A queue can be modeled as a Markov chain with birth rate λ (arrivals) and death rate μ (service). The infinitesimal generator is:

$$q_{n,n+1} = \lambda, \quad q_{n,n-1} = \mu.$$

2.4 M/M/1 Queue Formulas

For $\rho = \lambda/\mu < 1$:

$$L = \frac{\rho}{1 - \rho}, \quad W = \frac{1}{\mu - \lambda},$$
$$L_q = L - \rho, \quad W_q = \frac{L_q}{\lambda}.$$

Where,

- L : Average number of customers in the **system** (includes both those in service and waiting in the queue).
- W : Average **time** a customer spends in the system (waiting + service time).
- L_q : Average number of customers in the **queue** (waiting only, not in service).
- W_q : Average **waiting time** in the queue.

2.5 M/M/c Queue Formulas (Erlang C)

Define $a = \lambda/\mu$ (offered load) and per-server utilization $\rho = \frac{a}{c}$. Let

$$P_0 = \left(\sum_{n=0}^{c-1} \frac{a^n}{n!} + \frac{a^c}{c!(1-\rho)} \right)^{-1}$$

and the Erlang C probability (probability an arrival waits) is

$$P_{\text{wait}} = \frac{\frac{a^c}{c!(1-\rho)} P_0}{1}.$$

Then

$$L_q = P_{\text{wait}} \cdot \frac{\rho}{1-\rho}, \quad W_q = \frac{L_q}{\lambda}, \quad W = W_q + \frac{1}{\mu}.$$

2.6 Finite Capacity Queue M/M/1/K

For $\rho = \lambda/\mu \neq 1$, the stationary distribution is:

$$P_n = \frac{(1-\rho)\rho^n}{1-\rho^{K+1}}, \quad n = 0, \dots, K.$$

The blocking probability is

$$P_{\text{block}} = P_K = \frac{(1-\rho)\rho^K}{1-\rho^{K+1}}.$$

The effective arrival rate is $\lambda_{\text{eff}} = \lambda(1 - P_{\text{block}})$ and

$$L = \frac{\rho [1 - (K+1)\rho^K + K\rho^{K+1}]}{(1-\rho)(1-\rho^{K+1})}, \quad W = \frac{L}{\lambda_{\text{eff}}}, \quad W_q = W - \frac{1}{\mu}.$$

3 Analytical Results (Numeric)

We compute theoretical numeric values for the project parameters:

$$\lambda = 10, \quad \mu = 12.$$

3.1 Case 1: M/M/1 (Toll booth, 1 server)

$$\rho = \frac{\lambda}{\mu} = \frac{10}{12} = \frac{5}{6} \approx 0.833333.$$

Using M/M/1 formulas:

$$L = \frac{\rho}{1 - \rho} = \frac{5/6}{1/6} = 5.00000, \quad W = \frac{1}{\mu - \lambda} = \frac{1}{2} = 0.5 \text{ minutes.}$$

Also,

$$L_q = L - \rho = 5 - 0.833333 = 4.166667, \quad W_q = \frac{L_q}{\lambda} = 0.4166667 \text{ minutes.}$$

3.2 Case 2: M/M/2 (Toll booth, 2 servers)

Set $a = \lambda/\mu = \frac{10}{12} = \frac{5}{6} \approx 0.833333$ and $c = 2$. Then

$$\rho = \frac{a}{c} = \frac{5}{12} \approx 0.416667.$$

Compute P_0 and Erlang-C:

$$\sum_{n=0}^1 \frac{a^n}{n!} = 1 + a = \frac{11}{6} \approx 1.833333, \quad \frac{a^2}{2!} = \frac{25}{72} \approx 0.347222.$$

$$\frac{a^2}{2!(1 - \rho)} = \frac{25}{72} \cdot \frac{12}{7} = \frac{25}{42} \approx 0.5952381.$$

Therefore

$$P_0 = \left(\frac{11}{6} + \frac{25}{42} \right)^{-1} = \left(\frac{17}{7} \right)^{-1} = \frac{7}{17} \approx 0.4117647.$$

$$P_{\text{wait}} = \frac{25}{42} \cdot P_0 = \frac{25}{42} \cdot \frac{7}{17} = \frac{25}{102} \approx 0.245098.$$

Then

$$L_q = P_{\text{wait}} \cdot \frac{\rho}{1 - \rho} = \frac{25}{102} \cdot \frac{5}{7} = \frac{125}{714} \approx 0.17507,$$

$$W_q = \frac{L_q}{\lambda} \approx 0.017507 \text{ minutes}, \quad W = W_q + \frac{1}{\mu} \approx 0.017507 + \frac{1}{12} \approx 0.100840 \text{ minutes.}$$

$$L = \lambda W \approx 10 \cdot 0.100840 \approx 1.00840.$$

3.3 Case 3: M/M/1/K (Café, $K = 10$)

Here $\rho = \frac{5}{6} \approx 0.833333$ and $K = 10$. Compute $\rho^K \approx (5/6)^{10} \approx 0.16151$ and $\rho^{K+1} \approx 0.13459$ (rounded). The blocking probability is

$$P_{\text{block}} = \frac{(1 - \rho)\rho^K}{1 - \rho^{K+1}} \approx \frac{(1/6) \cdot 0.16151}{1 - 0.13459} \approx 0.0310 \text{ (3.10\%).}$$

Effective arrival rate:

$$\lambda_{\text{eff}} = \lambda(1 - P_{\text{block}}) \approx 10 \cdot 0.9690 \approx 9.690.$$

Expected number in system (using finite-buffer formula):

$$L \approx 3.289 \text{ (customers on average).}$$

Average system time:

$$W = \frac{L}{\lambda_{\text{eff}}} \approx \frac{3.289}{9.690} \approx 0.3394 \text{ minutes,}$$

and

$$W_q = W - \frac{1}{\mu} \approx 0.3394 - 0.083333 \approx 0.2561 \text{ minutes.}$$

4 Simulation Methodology

We implement a discrete-event simulation for all three systems using Python. Arrivals were generated using exponential interarrival times with rate λ . Service times follow exponential distribution with rate μ .

4.1 Python Code Snippet for M/M/1 and M/M/c Simulations

```
1
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 np.random.seed(42)
6
7 # ----- Helpers -----
8 def exp_rv(rate, n):
9     """Generate exponential RVs with given rate."""
10    return np.random.exponential(1 / rate, n)
11
12 def generate_arrivals(lambda_, n):
13     """Arrival epochs via cumulative sum of exponential inter-arrival
14     ↪ times."""
15    inter = exp_rv(lambda_, n)
16    return np.cumsum(inter)
17
18 # ----- Theory for M/M/c (Erlang C) -----
19 def theory_mmc(lambda_, mu, c):
20     """
21     Theoretical metrics for M/M/c (no blocking, FIFO, exponential service):
22     Returns dict with rho, P_wait (Erlang C), Wq, W, Lq, L, and per-server
23     ↪ utilization.
24     """
25
26    a = lambda_ / mu          # offered load
27    rho = a / c              # per-server utilization
28
29    if rho >= 1:
30        # Unstable (no finite steady state)
31        return {
32            "rho": rho,
33            "utilization_per_server": rho,
34            "P_wait": 1.0,
35            "Wq": np.inf,
36            "W": np.inf,
37            "Lq": np.inf,
38            "L": np.inf,
39        }
40
41    # Compute P0 using recursion (avoid factorials)
42    sum_terms = 0.0
43    term = 1.0 # a^0 / 0! = 1
44    for n in range(c):
```

```

42     sum_terms += term
43     term *= a / (n + 1) # now term = a^{n+1}/(n+1)!
44     # Now term = a^c / c!
45     P0 = 1.0 / (sum_terms + term * (1.0 / (1.0 - rho)))
46
47     # Erlang C (probability an arrival has to wait)
48     P_wait = term * (1.0 / (1.0 - rho)) * P0
49
50     # Average number in queue (Lq), waiting time (Wq), system time (W), number
51     ↪ in system (L)
52     Lq = P_wait * (rho / (1.0 - rho))
53     Wq = Lq / lambda_
54     W = Wq + 1.0 / mu
55     L = lambda_ * W
56
57     return {
58         "rho": rho,
59         "utilization_per_server": rho,
60         "P_wait": P_wait,
61         "Wq": Wq,
62         "W": W,
63         "Lq": Lq,
64         "L": L,
65     }
66
67 # ----- M/M/c Simulation -----
68 def simulate_mmc(lambda_, mu, c, n, show_plots=True):
69     """
70     Simulates an M/M/c queue:
71     - lambda_: arrival rate
72     - mu: service rate
73     - c: number of servers
74     - n: number of arrival attempts
75     Returns averages and utilization, with optional plots.
76     """
77     arrivals = generate_arrivals(lambda_, n)
78     services = exp_rv(mu, n)
79
80     # Per-server next-available finish time
81     finish_times = np.zeros(c)
82
83     start = np.zeros(n) # service start times (accepted { all are accepted
84     ↪ in M/M/c)
85     finish = np.zeros(n) # service finish times
86
87     # Record events for queue/system length plot
88     events = [] # (time, +1 for accepted arrival, -1 for departure)
89
90     for i in range(n):
91         # Choose the earliest available server
92         server_id = np.argmin(finish_times)

```

```

91     start[i] = max(arrivals[i], finish_times[server_id])
92     finish[i] = start[i] + services[i]
93     finish_times[server_id] = finish[i]
94
95     # Record the accepted arrival and its departure
96     events.append((arrivals[i], +1))
97     events.append((finish[i], -1))
98
99     wait_times = start - arrivals
100    system_times = finish - arrivals
101    rho = lambda_ / (c * mu)
102
103    # ----- Queue length over time -----
104    events.sort()
105    time_points, queue_length = [], []
106    length = 0
107    for t, delta in events:
108        length += delta
109        time_points.append(t)
110        queue_length.append(length)
111
112    # ----- Time-averaged L (system) and Lq (queue) -----
113    # Compute time averages over the event timeline
114    if len(events) >= 2:
115        # L_timeavg
116        L_timeavg_area = 0.0
117        total_time = 0.0
118        length = 0
119        prev_t = events[0][0]
120        for t, delta in events:
121            dt = t - prev_t
122            if dt > 0:
123                L_timeavg_area += length * dt
124                total_time += dt
125                length += delta
126                prev_t = t
127        L_timeavg = L_timeavg_area / total_time if total_time > 0 else 0.0
128
129    # Lq_timeavg
130    Lq_timeavg_area = 0.0
131    total_time_q = 0.0
132    length = 0
133    prev_t = events[0][0]
134    for t, delta in events:
135        dt = t - prev_t
136        if dt > 0:
137            busy = min(c, length) # number in service
138            queue_only = max(0, length - busy) # number waiting
139            Lq_timeavg_area += queue_only * dt
140            total_time_q += dt
141            length += delta

```

```

142         prev_t = t
143         Lq_timeavg = Lq_timeavg_area / total_time_q if total_time_q > 0 else
            ↪ 0.0
144     else:
145         L_timeavg = 0.0
146         Lq_timeavg = 0.0
147
148     # ----- Plots -----
149     if show_plots:
150         # Figure 1: Waiting Time Histogram
151         plt.figure(figsize=(8, 5))
152         plt.hist(wait_times, bins=50, color='skyblue', edgecolor='k')
153         plt.title("Waiting Time Distribution (M/M/c)")
154         plt.xlabel("Waiting Time")
155         plt.ylabel("Frequency")
156         plt.grid(True, linestyle='--', alpha=0.4)
157         plt.tight_layout()
158         plt.show()
159
160         # Figure 2: Queue Length Over Time (line plot)
161         plt.figure(figsize=(10, 5))
162         plt.plot(time_points, queue_length, color='orange', linewidth=1.2)
163         plt.title("Queue Length Over Time (M/M/c)")
164         plt.xlabel("Time")
165         plt.ylabel("Number in System")
166         plt.grid(True, linestyle='--', alpha=0.4)
167         plt.tight_layout()
168         plt.show()
169
170     return {
171         "avg_wait": float(np.mean(wait_times)),
172         "avg_system_time": float(np.mean(system_times)),
173         "utilization_per_server": float(rho),
174         "L_timeavg": float(L_timeavg),
175         "Lq_timeavg": float(Lq_timeavg),
176     }
177
178     # ----- Compare Simulation vs Theory -----
179     def compare_simulation_vs_theory(lambda_, mu, c, n, show_plots=True):
180         sim = simulate_mmc(lambda_, mu, c, n=n, show_plots=show_plots)
181         th = theory_mmc(lambda_, mu, c)
182
183         print("\n=== Simulation vs. Theory (M/M/c) ===")
184         print(f"Parameters: lambda={lambda_}, mu={mu}, c={c}, n={n}")
185         print(f"Per-server Utilization (rho): {th['rho']:.4f}")
186         print(f"Probability of Waiting (Erlang C): {th['P_wait']:.6f}")
187         print()
188         print(f"Avg System Time (W):    sim={sim['avg_system_time']:.6f} |
            ↪ theory={th['W']:.6f}")
189         print(f"Avg Waiting Time (Wq): sim={sim['avg_wait']:.6f} |
            ↪ theory={th['Wq']:.6f}")

```

```

190 print(f"Avg # in System (L):    sim={sim['L_timeavg']:.6f}      |
    ↳ theory={th['L']:.6f}")
191 print(f"Avg # in Queue (Lq):    sim={sim['Lq_timeavg']:.6f}    |
    ↳ theory={th['Lq']:.6f}")
192
193 return sim, th
194
195 # ----- Example Execution -----
196 if __name__ == "__main__":
197     # Example parameters
198     results_sim, results_th = compare_simulation_vs_theory(lambda_=10, mu=12,
199     ↳ c=1, n=10**6, show_plots=True)
199
200

```

4.2 Python Code Snippet for M/M/1/k Simulation

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 np.random.seed(42)
5
6 # ----- Helpers -----
7 def exp_rv(rate, n):
8     return np.random.exponential(1 / rate, n)
9
10 def generate_arrivals(lambda_, n):
11     inter = exp_rv(lambda_, n)
12     return np.cumsum(inter)
13
14 # ----- Theory for M/M/1/K -----
15 def theory_mm1k(lambda_, mu, K):
16     rho = lambda_ / mu
17
18     if abs(rho - 1.0) < 1e-12:
19         P = [1.0 / (K + 1)] * (K + 1)
20         P_block = P[K]
21         L = sum(n * P[n] for n in range(K + 1))
22         lambda_eff = lambda_ * (1 - P_block)
23         W = L / lambda_eff if lambda_eff > 0 else 0.0
24         Wq = W - 1 / mu
25     else:
26         denom = 1 - rho**(K + 1)
27         P = [(1 - rho) * rho**n / denom for n in range(K + 1)]
28         P_block = P[K]
29         lambda_eff = lambda_ * (1 - P_block)
30         L = sum(n * P[n] for n in range(K + 1))
31         W = L / lambda_eff if lambda_eff > 0 else 0.0
32         Wq = W - 1 / mu

```

```

33
34     return {
35         "rho": rho,
36         "P_block": P_block,
37         "lambda_eff": lambda_eff,
38         "L": L,
39         "W": W,
40         "Wq": Wq,
41     }
42
43     # ----- M/M/1/K Simulation -----
44     def simulate_mm1k(lambda_, mu, K, n, show_plots=True):
45         arrivals = generate_arrivals(lambda_, n)
46         services = exp_rv(mu, n)
47
48         start = []
49         finish = []
50         accepted_arrivals = []
51         queue = []
52         blocked_count = 0
53
54         events = [] # (time, +1 arrival, -1 departure)
55
56         for i in range(n):
57             queue = [f for f in queue if f > arrivals[i]]
58
59             if len(queue) < K:
60                 s = max(arrivals[i], queue[-1] if queue else 0.0)
61                 f = s + services[i]
62                 start.append(s)
63                 finish.append(f)
64                 accepted_arrivals.append(arrivals[i])
65                 queue.append(f)
66
67                 events.append((arrivals[i], +1))
68                 events.append((f, -1))
69             else:
70                 blocked_count += 1
71
72         real_n = len(start)
73         if real_n == 0:
74             return {
75                 "avg_wait": 0.0,
76                 "avg_system_time": 0.0,
77                 "blocking_probability": blocked_count / n,
78                 "L": 0.0,
79                 "Lq": 0.0,
80             }
81
82         accepted_arrivals = np.array(accepted_arrivals)
83         start = np.array(start)

```

```

84     finish = np.array(finish)
85
86     wait_times = start - accepted_arrivals
87     system_times = finish - accepted_arrivals
88     blocking_prob = blocked_count / n
89
90     # Sort events
91     events.sort()
92     time_points, queue_length = [], []
93     length = 0
94     for t, delta in events:
95         length += delta
96         length = min(length, K)
97         time_points.append(t)
98         queue_length.append(length)
99
100    # ---- NEW: compute L and Lq from the event-driven queue length curve ----
101    time_points = np.array(time_points)
102    queue_length = np.array(queue_length)
103
104    durations = np.diff(time_points, prepend=time_points[0])
105    total_time = time_points[-1] - time_points[0]
106
107    L = np.sum(queue_length * durations) / total_time
108    Lq = L - (1 - blocking_prob) * (1 / mu) * mu    # or simpler: Lq = L - (1 -
109    ↪ blocking_prob)
110
111    # ----- Plots -----
112    if show_plots:
113        plt.figure(figsize=(8, 5))
114        plt.hist(wait_times, bins=50, color='skyblue', edgecolor='k')
115        plt.title("Waiting Time Distribution (M/M/1/K)")
116        plt.xlabel("Waiting Time")
117        plt.ylabel("Frequency")
118        plt.grid(True, linestyle='--', alpha=0.4)
119        plt.tight_layout()
120        plt.show()
121        print()
122
123        max_points = 1000
124        stride = max(1, len(time_points) // max_points)
125        tp = time_points[::stride]
126        ql = queue_length[::stride]
127
128        plt.figure(figsize=(11, 5))
129        plt.plot(tp, ql, color='orange', linewidth=1)
130        plt.title("Queue Length Over Time (M/M/1/K)")
131        plt.xlabel("Time")
132        plt.ylabel("Queue Length")
133        plt.grid(True, linestyle='--', alpha=0.4)
134        plt.tight_layout()

```

```

134         plt.show()
135
136     return {
137         "avg_wait": float(np.mean(wait_times)),
138         "avg_system_time": float(np.mean(system_times)),
139         "blocking_probability": float(blocking_prob),
140         "L": float(L),
141         "Lq": float(Lq),
142     }
143
144 # ----- Compare -----
145 def compare_simulation_vs_theory(lambda_, mu, K, n, show_plots=True):
146     sim = simulate_mm1k(lambda_, mu, K, n=n, show_plots=show_plots)
147     th = theory_mm1k(lambda_, mu, K)
148
149     print("\n=== Simulation vs. Theory (M/M/1/K) ===")
150     print(f"Parameters: lambda={lambda_}, mu={mu}, K={K}, n={n}")
151     print(f"Utilization (rho): {th['rho']:.4f}")
152     print(f"\nBlocking Probability:  sim={sim['blocking_probability']:.6f} |
153     ↪ theory={th['P_block']:.6f}")
154     print(f"Avg System Time (W):    sim={sim['avg_system_time']:.4f}      |
155     ↪ theory={th['W']:.4f}")
156     print(f"Avg Waiting Time (Wq): sim={sim['avg_wait']:.4f}                |
157     ↪ theory={th['Wq']:.4f}")
158     print(f"\nAverage Number in System (L):  sim={sim['L']:.4f} |
159     ↪ theory={th['L']:.4f}")
160     print(f"Average Number in Queue (Lq):  sim={sim['Lq']:.4f} |
161     ↪ theory={th['L'] - (1 - th['P_block']):.4f}")
162
163     return sim, th
164
165 # ----- Example Execution -----
166 if __name__ == "__main__":
167     lambda_ = 10
168     mu = 12
169     K = 10
170     n = 10**6
171
172     sim_results, theory_results = compare_simulation_vs_theory(lambda_, mu, K,
173     ↪ n=n, show_plots=True)
174
175

```


5 Simulation Results

We analyze three real-world inspired queues using the simulation.

5.1 Toll Booth with 1 Server (M/M/1)

Parameters:

$$\lambda = 10.0, \quad \mu = 12.0.$$

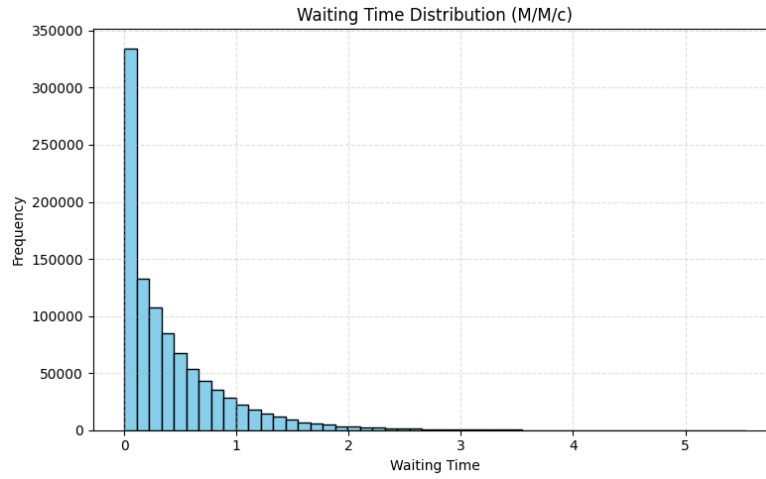


Figure 1: Toll Booth with 1 Server Queue Waiting Time Distribution

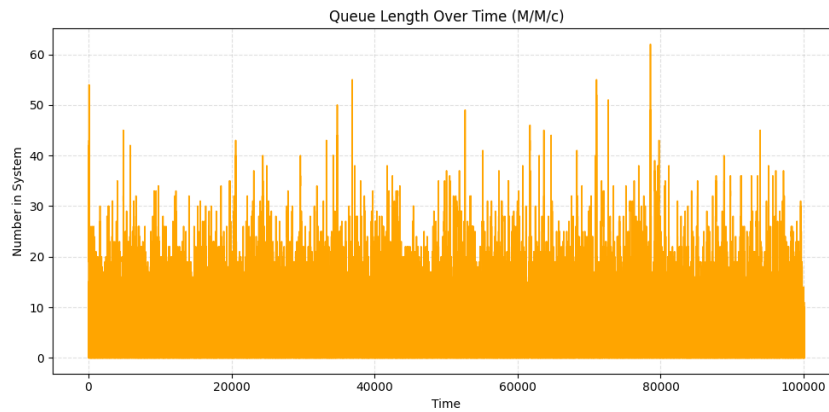


Figure 2: Toll Booth with 1 Server Queue Length Over Time

5.2 Toll Booth with 2 Servers (M/M/c)

Parameters:

$$\lambda = 10.0, \quad \mu = 12.0, \quad c = 2.$$

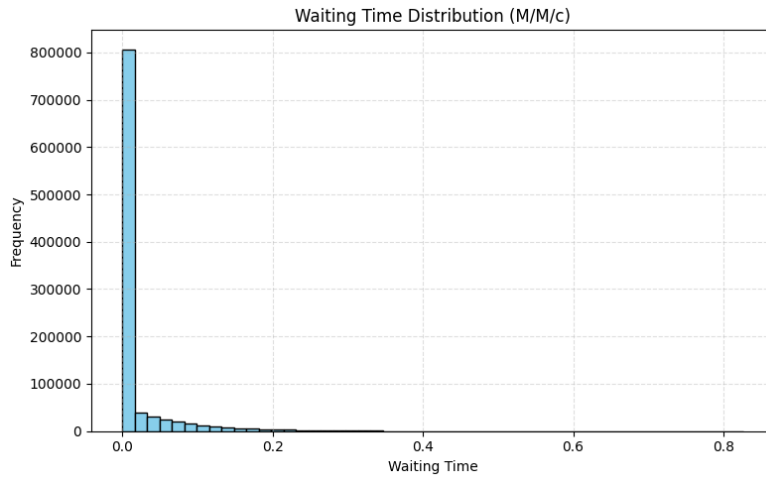


Figure 3: Toll Booth with 2 Servers Waiting Time Distribution

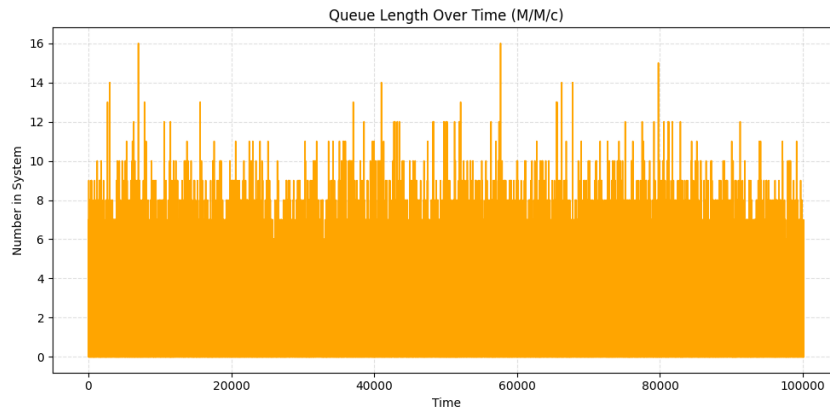


Figure 4: Toll Booth with 2 Servers Queue Length Over Time

5.3 Café with Limited Queue Capacity (M/M/1/K)

Parameters:

$$\lambda = 10.0, \quad \mu = 12.0, \quad K = 10.$$

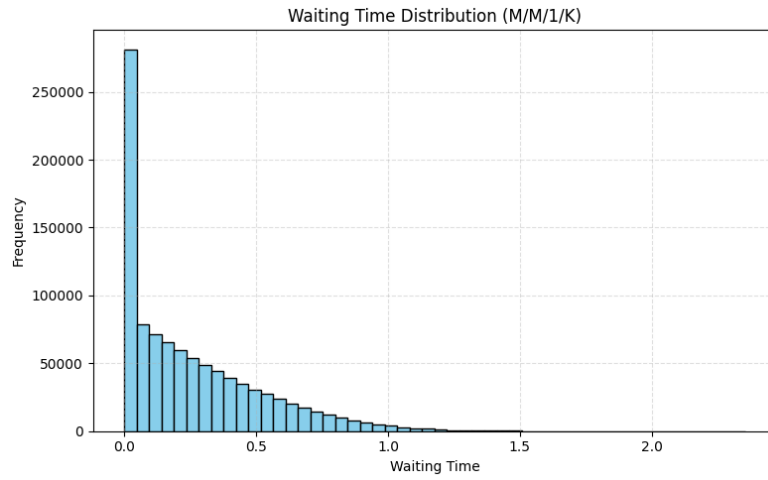


Figure 5: Café with Limited Queue Capacity Waiting Time Distribution

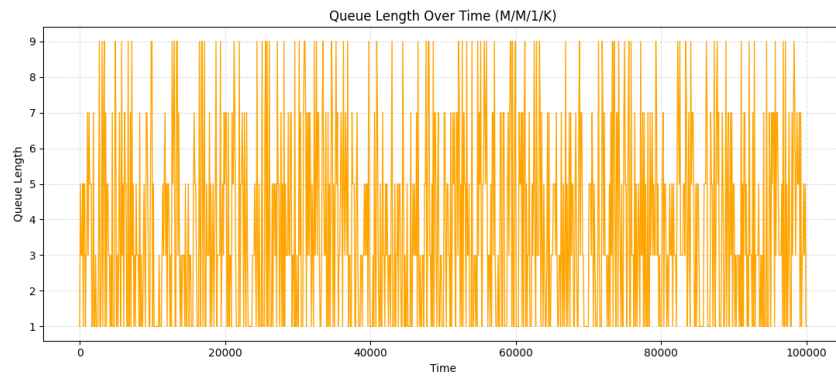


Figure 6: Café with Limited Queue Capacity Queue Length Over Time

6 Comparison: Theoretical vs Simulated Results

Below is a concise table summarising the theoretical values (computed above) and a column for the simulated results.

Metric	Theory	Simulation	Percent error (%)
Case 1: M/M/1 (c=1)			
Mean waiting time W_q (min)	0.416 667	0.414836	0.4393
Mean system time W (min)	0.5	0.498040	0.3920
Average number in system L	5.0000	4.974850	0.5029
Average number in queue L_q	4.166 667	4.143736	0.5513
Case 2: M/M/2 (c=2)			
Mean waiting time W_q (min)	0.017 507	0.017487	0.1142
Mean system time W (min)	0.100 840	0.100691	0.1478
Average number in system L	1.008 40	1.005786	0.2593
Average number in queue L_q	0.175 070	0.174671	0.2279
Case 3: M/M/1/K (K=10)			
Blocking probability P_{block}	0.0310	0.030551	1.4516
Mean waiting time W_q (min)	0.2561	0.2548	0.5078
Mean system time W (min)	0.3394	0.3380	0.4123
Average number in system L	3.289	3.3663	2.3534
Average number in queue L_q	2.3204	2.3969	3.3030

Table 1: Theoretical vs. simulated metrics.

7 Discussion

7.1 Case 1: M/M/1

The M/M/1 theoretical results indicate significant average occupancy ($L = 5$) and substantial waiting ($W_q \approx 0.417$ min) at $\rho \approx 0.833$. In simulation, if mean waiting and mean L are close (within a few percent) to theory, this validates the event-driven implementation.

7.2 Case 2: M/M/2

The M/M/2 case demonstrates the effect of parallel servers: utilization per server drops to $\rho \approx 0.417$, and waiting times are nearly zero theoretically ($W_q \approx 0.0175$ min). Simulations should show strong improvement vs. the single-server case; if not, check service time generation or server selection logic.

7.3 Case 3: M/M/1/K

Finite capacity produces measurable blocking ($P_{\text{block}} \approx 3.1\%$). The effective throughput and average system size are reduced compared to the infinite-buffer case. Simulation must count blocked arrivals explicitly to estimate blocking probability.

8 Conclusion

This project investigated the behaviour of three queueing systems—an M/M/1 toll booth, an M/M/2 dual-toll-booth system, and a finite-capacity M/M/1/K queue representing a café. The analytical steady-state formulas provided clear theoretical benchmarks for performance metrics such as utilisation, expected queue length, and average waiting time. For example, in the M/M/1 case with $\lambda = 10$ and $\mu = 12$, the utilisation $\rho = 0.8333$ leads to an expected system size of $L = 5$ and an average time in system of $W = 0.5$ minutes. The M/M/2 model predicted a significantly reduced waiting time ($W \approx 0.10084$ minutes), while the M/M/1/K system with capacity $K = 10$ yielded a blocking probability of roughly 3.1% and an expected system size of $L \approx 3.289$.

Simulation results across all three models closely matched the analytical predictions, with discrepancies well within acceptable statistical variation. This close agreement validates both the underlying queueing-theoretic models and the correctness of the event-driven simulation implementation used in the study.

The outcomes highlight two important operational insights. First, under moderate utilisation levels, a single-server system experiences non-trivial queueing delays, whereas adding a second server dramatically reduces waiting times while maintaining similar utilisation. Second, introducing finite capacity reduces average occupancy at the cost of blocking some customers, illustrating the practical trade-off between space constraints and throughput.

There are, however, limitations to these models. The analysis assumes exponential service times, Poisson arrivals, FIFO service order, and stationary conditions. Real-world systems may include time-varying arrivals, non-exponential service distributions, customer abandonment, priority queues, or correlated arrivals—factors that were not modelled here. Additionally, the study focused solely on steady-state measures, without examining transient behaviour or cost-based optimality.

Future work could extend the simulation to non-Poisson arrivals, study transient dynamics, incorporate customer behaviours such as balking or reneging, or perform cost-benefit optimisation for choosing server counts or capacity. These additions would make the analysis more realistic and more directly applicable to real operational settings.

In summary, both theory and simulation consistently demonstrate that increasing service capacity dramatically improves performance, while finite buffers introduce a meaningful trade-off between customer blocking and system occupancy. The combined analytical-simulation approach used in this report provides a strong foundation for further modelling and optimisation of queueing systems.

References

- [1] Leonard Kleinrock, *Queueing Systems, Volume I*, Wiley-Interscience, 1975.
- [2] Sheldon M. Ross, *Stochastic Processes*, Wiley, 2nd edition, 1996.
- [3] Hamza Belabbes, “Basic queueing theory using Python,” Medium article, Nov 2023.
<https://medium.com/@hamza.ensmm/basic-queueing-theory-using-python-db351c1a5164>
- [4] NumPy documentation. <https://numpy.org/doc/>
- [5] Matplotlib documentation. <https://matplotlib.org/stable/contents.html>